# ADVANCED SQL TRAINING

Corporate Training Series — training_db

# Answer Key

*Complete SQL Solutions — 100 Questions*

Intermediate reference solutions — multiple approaches may be valid

E-Commerce Schema | MySQL 8+ | 100 Questions

# Notes on Answers

All answers use the training_db schema. Queries are tested for MySQL 8+. Comments in queries explain key concepts. Alternative approaches (e.g., JOIN instead of subquery) are often valid — what matters is correctness and readability.

## Section 1 — SELECT, WHERE & Operators

### Q1. [Basic] *SELECT – All Columns*

*Write a query to display all columns and all rows from the `customers` table.*

```
SELECT * FROM customers;
```

### Q2. [Basic] *SELECT – Specific Columns*

*Write a query to display only the `name` and `city` columns from the `customers` table.*

```
SELECT name, city FROM customers;
```

### Q3. [Basic] *WHERE – Equality*

*List all customers who live in 'Mumbai'.*

```
SELECT * FROM customers
WHERE city = 'Mumbai';
```

### Q4. [Basic] *WHERE – Inequality*

*List all products that are NOT in the 'Electronics' category.*

```
SELECT * FROM products
WHERE category <> 'Electronics';
```

### Q5. [Basic] *WHERE – Numeric Comparison*

*Find all products whose price is greater than ₹10,000.*

```
SELECT product_id, name, price
FROM products
WHERE price > 10000;
```

### Q6. [Basic] *ORDER BY – ASC*

*Display all products sorted by price from lowest to highest.*

```
SELECT * FROM products
ORDER BY price ASC;
```

### Q7. [Basic] *ORDER BY – DESC*

*Display all orders sorted by order_date from newest to oldest.*

```
SELECT * FROM orders
ORDER BY order_date DESC;
```

### Q8. [Basic] *DISTINCT*

*List all unique cities from which customers have registered.*

```
SELECT DISTINCT city
FROM customers;
```

### Q9. [Basic] *LIMIT*

*Show the top 3 most expensive products.*

```
SELECT name, price
FROM products
ORDER BY price DESC
LIMIT 3;
```

### Q10. [Basic] *BETWEEN*

*Find all products priced between ₹2,000 and ₹15,000 (inclusive).*

```
SELECT name, category, price
FROM products
WHERE price BETWEEN 2000 AND 15000;
```

### Q11. [Basic] *IN*

*List all customers who live in Mumbai, Delhi, or Bangalore.*

```
SELECT name, city
FROM customers
WHERE city IN ('Mumbai', 'Delhi', 'Bangalore');
```

### Q12. [Basic] *NOT IN*

*Find all orders whose status is neither 'cancelled' nor 'pending'.*

```
SELECT order_id, status, total
FROM orders
WHERE status NOT IN ('cancelled', 'pending');
```

### Q13. [Basic] *LIKE – Prefix*

*Find all customers whose name starts with the letter 'A'.*

```
SELECT * FROM customers
WHERE name LIKE 'A%';
```

### Q14. [Basic] *LIKE – Contains*

*Find all products whose name contains the word 'Chair'.*

```
SELECT * FROM products
WHERE name LIKE '%Chair%';
```

### Q15. [Basic] *LIKE – Single Character*

*Find all customers whose name has exactly 8 characters.*

```
SELECT name FROM customers
WHERE name LIKE '________';  -- eight underscores
```

### Q16. [Basic] *IS NULL*

*Find all employees who do NOT have a manager (i.e., manager_id is NULL).*

```
SELECT emp_id, name, dept
FROM employees
WHERE manager_id IS NULL;
```

### Q17. [Basic] *IS NOT NULL*

*Find all employees who DO have a manager.*

```
SELECT emp_id, name, manager_id
FROM employees
WHERE manager_id IS NOT NULL;
```

### Q18. [Basic] *AND*

*Find all Electronics products with stock greater than 50.*

```
SELECT name, price, stock
FROM products
WHERE category = 'Electronics'
  AND stock > 50;
```

### Q19. [Basic] *OR*

*Find all orders that are either 'delivered' or have a total greater than ₹20,000.*

```
SELECT order_id, status, total
FROM orders
WHERE status = 'delivered'
   OR total > 20000;
```

### Q20. [Basic] *NOT*

*List all products that are not in the 'Furniture' category and are priced under ₹5,000.*

```
SELECT name, category, price
```

```
FROM products
WHERE NOT category = 'Furniture'
  AND price < 5000;
```

## Section 2 — Aggregate Functions & GROUP BY

### Q21. [Basic] *COUNT(*)*

*Count the total number of customers in the database.*

```
SELECT COUNT(*) AS total_customers
FROM customers;
```

### Q22. [Basic] *COUNT with WHERE*

*Count how many orders have the status 'delivered'.*

```
SELECT COUNT(*) AS delivered_orders
FROM orders
WHERE status = 'delivered';
```

### Q23. [Basic] *SUM*

*Find the total revenue from all orders combined.*

```
SELECT SUM(total) AS total_revenue
FROM orders;
```

### Q24. [Basic] *AVG*

*Find the average price of all products.*

```
SELECT ROUND(AVG(price), 2) AS avg_price
FROM products;
```

### Q25. [Basic] *MIN / MAX*

*Find the cheapest and most expensive product prices.*

```
SELECT MIN(price) AS cheapest,
       MAX(price) AS most_expensive
FROM products;
```

### Q26. [Intermediate] *GROUP BY*

*Count the number of products in each category.*

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category
ORDER BY product_count DESC;
```

### Q27. [Intermediate] *GROUP BY + SUM*

*Find the total stock available for each product category.*

```sql
SELECT category, SUM(stock) AS total_stock
FROM products
GROUP BY category;
```

### Q28. [Intermediate] *GROUP BY + AVG*

*Find the average order total for each order status.*

```sql
SELECT status,
       ROUND(AVG(total), 2) AS avg_order_value,
       COUNT(*)             AS order_count
FROM orders
GROUP BY status;
```

### Q29. [Intermediate] *HAVING*

*List product categories that have more than 1 product.*

```sql
SELECT category, COUNT(*) AS cnt
FROM products
GROUP BY category
HAVING COUNT(*) > 1;
```

### Q30. [Intermediate] *WHERE + GROUP BY + HAVING*

*Among orders placed in 2023, find customers who have placed more than 1 order.*

```sql
SELECT customer_id, COUNT(*) AS num_orders
FROM orders
WHERE YEAR(order_date) = 2023
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

## Section 3 — INNER JOIN

### Q31. [Basic] *INNER JOIN – 2 Tables*

*List all orders along with the name of the customer who placed each order.*

```
SELECT o.order_id, c.name AS customer_name,
       o.order_date, o.total
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date;
```

### Q32. [Basic] *INNER JOIN – Filter*

*Find all delivered orders and display the customer's name, city, and order total.*

```
SELECT c.name, c.city, o.order_id, o.total
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'delivered'
ORDER BY o.total DESC;
```

### Q33. [Intermediate] *INNER JOIN – 3 Tables*

*List all order line items showing customer name, product name, quantity, and unit price.*

```
SELECT c.name  AS customer,
       p.name  AS product,
       oi.qty,
       oi.unit_price
FROM order_items oi
INNER JOIN orders   o  ON oi.order_id   = o.order_id
INNER JOIN customers c ON o.customer_id  = c.customer_id
INNER JOIN products  p ON oi.product_id  = p.product_id
ORDER BY c.name, p.name;
```

### Q34. [Intermediate] *INNER JOIN – 4 Tables + Aggregate*

*For each customer, calculate the total quantity of items ordered across all their orders.*

```
SELECT c.name, SUM(oi.qty) AS total_items_ordered
FROM customers c
INNER JOIN orders      o  ON c.customer_id  = o.customer_id
INNER JOIN order_items oi ON o.order_id     = oi.order_id
GROUP BY c.customer_id, c.name
ORDER BY total_items_ordered DESC;
```

### Q35. [Intermediate] *INNER JOIN + GROUP BY*

*Find the total revenue generated by each product (qty × unit_price summed).*

```
SELECT p.name AS product, p.category,
       SUM(oi.qty * oi.unit_price) AS revenue
```

```
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name, p.category
ORDER BY revenue DESC;
```

## Section 4 — LEFT JOIN

### Q36. [Basic] *LEFT JOIN – All Rows*

*List all customers and their orders. Also show customers who have placed no orders (show NULL for order columns).*

```
SELECT c.name, c.city,
       o.order_id, o.order_date, o.total
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.name;
```

### Q37. [Basic] *LEFT JOIN – Find Unmatched*

*Find all customers who have NEVER placed any order.*

```
SELECT c.customer_id, c.name, c.city
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

### Q38. [Intermediate] *LEFT JOIN – Products Not Ordered*

*Find all products that have never been included in any order.*

```
SELECT p.product_id, p.name, p.category, p.price
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.item_id IS NULL;
```

### Q39. [Intermediate] *LEFT JOIN + COUNT*

*For each customer, show how many orders they have placed (include customers with 0 orders).*

```
SELECT c.name,
       COUNT(o.order_id) AS num_orders
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY num_orders DESC;
```

### Q40. [Intermediate] *LEFT JOIN + COALESCE*

*List all customers with their total spending. Show 0 for customers who haven't spent anything (use COALESCE).*

```
SELECT c.name,
       COALESCE(SUM(o.total), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC;
```

## Section 5 — RIGHT JOIN & FULL OUTER JOIN

### Q41. [Intermediate] *RIGHT JOIN*

*List all orders and the customer details for each. Include orders where the customer record may be missing.*

```
SELECT o.order_id, o.total, o.status,
       c.name, c.city
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

### Q42. [Intermediate] *FULL OUTER JOIN (UNION trick)*

*Write a query that shows ALL customers and ALL orders — highlighting unmatched rows on either side.*

```
SELECT c.name AS customer_name, o.order_id, o.total
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
UNION
SELECT c.name, o.order_id, o.total
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

## Section 6 — SELF JOIN

### Q43. [Intermediate] *SELF JOIN – Hierarchy*

*List each employee along with their manager's name. For top-level employees (no manager), show NULL.*

```
SELECT e.emp_id,
       e.name          AS employee_name,
       e.dept,
       e.salary,
       m.name          AS manager_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

### Q44. [Intermediate] *SELF JOIN – Salary Comparison*

*Find all employees who earn MORE than their direct manager.*

```
SELECT e.name   AS employee,  e.salary AS emp_salary,
       m.name   AS manager,   m.salary AS mgr_salary
FROM employees e
INNER JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

### Q45. [Intermediate] *SELF JOIN – Same Department*

*List all pairs of employees who work in the same department (no duplicates, no self-pairing).*

```
SELECT e1.name AS employee1, e2.name AS employee2, e1.dept
FROM employees e1
INNER JOIN employees e2
   ON e1.dept = e2.dept
  AND e1.emp_id < e2.emp_id   -- avoids (A,B) and (B,A) duplicates
ORDER BY e1.dept, e1.name;
```

## Section 7 — Scalar & IN Subqueries

**Q46.** **[Intermediate]** *Scalar Subquery – WHERE*

*Find all products priced above the average product price.*

```
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

**Q47.** **[Intermediate]** *Scalar Subquery – SELECT*

*Show each product's name, price, and the difference between its price and the overall average price.*

```
SELECT name, price,
       ROUND(price - (SELECT AVG(price) FROM products), 2) AS diff_from_avg
FROM products
ORDER BY diff_from_avg DESC;
```

**Q48.** **[Intermediate]** *Scalar Subquery – HAVING*

*Find all order statuses where the average order total exceeds the overall average order total.*

```
SELECT status, ROUND(AVG(total), 2) AS avg_total
FROM orders
GROUP BY status
HAVING AVG(total) > (SELECT AVG(total) FROM orders);
```

**Q49.** **[Intermediate]** *Scalar Subquery – Most Recent*

*Retrieve the details of the single most recent order placed.*

```
SELECT *
FROM orders
WHERE order_date = (SELECT MAX(order_date) FROM orders);
```

**Q50.** **[Intermediate]** *Scalar Subquery – MIN Salary*

*Find the employee(s) who earn the minimum salary in the company.*

```
SELECT emp_id, name, dept, salary
FROM employees
WHERE salary = (SELECT MIN(salary) FROM employees);
```

**Q51.** **[Intermediate]** *IN Subquery*

*Find all customers who have placed at least one order.*

```
SELECT customer_id, name, city
FROM customers
WHERE customer_id IN (
    SELECT DISTINCT customer_id FROM orders
```

```
);
```

### Q52. [Intermediate] *NOT IN Subquery*

*Find all products that appear in NO order_items rows (unsold inventory) using NOT IN.*

```
SELECT product_id, name, category, price
FROM products
WHERE product_id NOT IN (
    SELECT DISTINCT product_id FROM order_items
);
```

### Q53. [Intermediate] *IN Subquery – Nested 2 Levels*

*Find all customers who ordered at least one Electronics product.*

```
SELECT DISTINCT c.customer_id, c.name
FROM customers c
WHERE c.customer_id IN (
    SELECT o.customer_id
    FROM orders o
    WHERE o.order_id IN (
        SELECT oi.order_id
        FROM order_items oi
        JOIN products p ON oi.product_id = p.product_id
        WHERE p.category = 'Electronics'
    )
);
```

## Section 8 — EXISTS & Derived Table

### Q54. [Intermediate] *EXISTS*

*List all customers who have placed at least one order — using EXISTS.*

```
SELECT c.customer_id, c.name, c.city
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

### Q55. [Intermediate] *NOT EXISTS*

*List customers who have NEVER placed any order — using NOT EXISTS.*

```
SELECT c.customer_id, c.name, c.city
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

### Q56. [Intermediate] *EXISTS – Conditional*

*Find all products for which there is at least one order item with qty >= 2.*

```
SELECT p.product_id, p.name, p.price
FROM products p
WHERE EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
      AND oi.qty >= 2
);
```

### Q57. [Intermediate] *Derived Table*

*Using a derived table, find the top 3 customers by total spend.*

```
SELECT dt.name, dt.total_spent
FROM (
    SELECT c.name, SUM(o.total) AS total_spent
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.name
) AS dt
ORDER BY dt.total_spent DESC
LIMIT 3;
```

### Q58. [Intermediate] *Derived Table – Category Revenue*

*Using a derived table, find all product categories whose total revenue (qty × unit_price) exceeds ₹50,000.*

```
SELECT cat_revenue.category, cat_revenue.revenue
FROM (
    SELECT p.category,
           SUM(oi.qty * oi.unit_price) AS revenue
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    GROUP BY p.category
) AS cat_revenue
WHERE cat_revenue.revenue > 50000
ORDER BY cat_revenue.revenue DESC;
```

### Q59. [Intermediate] *Derived Table – Average of Aggregates*

*Find the average number of items per order across all orders (use a derived table to first count per order).*

```
SELECT ROUND(AVG(items_per_order), 2) AS avg_items
FROM (
    SELECT order_id, SUM(qty) AS items_per_order
    FROM order_items
    GROUP BY order_id
) AS order_summary;
```

## Section 9 — Correlated Subqueries [Focus]

**Q60.** **[Intermediate]** *Correlated Subquery – Category Avg*

*Find all products whose price is above the average price of their OWN category.*

```
SELECT p1.name, p1.category, p1.price
FROM products p1
WHERE p1.price > (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.category = p1.category
)
ORDER BY p1.category, p1.price DESC;
```

**Q61.** **[Intermediate]** *Correlated Subquery – Highest Per Group*

*Find the most expensive product in EACH category using a correlated subquery.*

```
SELECT p1.name, p1.category, p1.price
FROM products p1
WHERE p1.price = (
    SELECT MAX(p2.price)
    FROM products p2
    WHERE p2.category = p1.category
)
ORDER BY p1.category;
```

**Q62.** **[Intermediate]** *Correlated Subquery – Employee Dept Avg*

*List employees whose salary is above the average salary in their department.*

```
SELECT e1.name, e1.dept, e1.salary
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.dept = e1.dept
)
ORDER BY e1.dept, e1.salary DESC;
```

**Q63.** **[Intermediate]** *Correlated Subquery – Customer Last Order*

*For each customer, display their most recent order date using a correlated subquery in SELECT.*

```
SELECT c.name,
       c.city,
       (SELECT MAX(o.order_date)
        FROM orders o
        WHERE o.customer_id = c.customer_id) AS last_order_date
FROM customers c
ORDER BY last_order_date DESC;
```

### Q64. [Intermediate] *Correlated Subquery – Nth Salary*

*Using a correlated subquery, find the second highest salary in the employees table.*

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
    SELECT MAX(salary) FROM employees
);
```

### Q65. [Intermediate] *Correlated Subquery – Row Rank*

*List each order with its rank by total (highest = 1) within the same customer, using a correlated subquery.*

```
SELECT o1.order_id,
       o1.customer_id,
       o1.total,
       (SELECT COUNT(*)
        FROM orders o2
        WHERE o2.customer_id = o1.customer_id
          AND o2.total >= o1.total) AS rank_within_customer
FROM orders o1
ORDER BY o1.customer_id, rank_within_customer;
```

## Section 10 — Set Operations

### Q66. [Intermediate] *UNION*

*Produce a single list of all city names from customers and all dept names from employees, with no duplicates.*

```
SELECT city AS location FROM customers
UNION
SELECT dept  AS location FROM employees
ORDER BY location;
```

### Q67. [Intermediate] *UNION ALL*

*Show a combined list of order IDs from orders with status 'pending' and orders with total > ₹10,000 (include duplicates).*

```
SELECT order_id FROM orders WHERE status = 'pending'
UNION ALL
SELECT order_id FROM orders WHERE total > 10000
ORDER BY order_id;
```

### Q68. [Intermediate] *INTERSECT (using JOIN)*

*Find customer_ids that appear in BOTH the customers table and as a customer_id in the orders table (INTERSECT pattern using JOIN).*

```
-- Using INNER JOIN (works on all MySQL versions)
SELECT DISTINCT c.customer_id
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;

-- MySQL 8.0.31+ native syntax:
-- SELECT customer_id FROM customers
-- INTERSECT
-- SELECT customer_id FROM orders;
```

### Q69. [Intermediate] *EXCEPT/MINUS (using NOT IN)*

*Find all customer_ids in the customers table that do NOT appear in the orders table (EXCEPT pattern).*

```
-- Using NOT IN (all MySQL versions)
SELECT customer_id FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders
);

-- MySQL 8.0.31+ native syntax:
-- SELECT customer_id FROM customers
-- EXCEPT
-- SELECT customer_id FROM orders;
```

## Section 11 — Aliases & Expressions

### Q70. [Basic] *Column Alias – Calculated*

*Display each product's name, price, and price with 18% GST added. Label the GST column as `price_with_gst`.*

```
SELECT name,
       price,
       ROUND(price * 1.18, 2) AS price_with_gst
FROM products
ORDER BY price_with_gst DESC;
```

### Q71. [Basic] *Table Alias*

*Rewrite the query: 'List all orders with the customer name' — using single-letter table aliases.*

```
SELECT o.order_id, c.name, o.order_date, o.total
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;
```

### Q72. [Intermediate] *Alias in ORDER BY*

*Compute each employee's annual salary (monthly × 12) and sort by it descending. Alias as `annual_salary`.*

```
SELECT name, dept,
       salary         AS monthly_salary,
       salary * 12    AS annual_salary
FROM employees
ORDER BY annual_salary DESC;
```

## Section 12 — Views & Indexes

### Q73. [Intermediate] *CREATE VIEW*

*Create a view named `vw_customer_orders` that shows customer name, city, order_id, order_date, and total for all orders.*

```
CREATE VIEW vw_customer_orders AS
SELECT c.name       AS customer_name,
       c.city,
       o.order_id,
       o.order_date,
       o.total
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;

-- Use the view:
SELECT * FROM vw_customer_orders
ORDER BY order_date DESC;
```

### Q74. [Intermediate] *VIEW + WHERE*

*Query the view `vw_customer_orders` to show only orders with total > ₹10,000.*

```
SELECT * FROM vw_customer_orders
WHERE total > 10000
ORDER BY total DESC;
```

### Q75. [Intermediate] *CREATE OR REPLACE VIEW*

*Modify the view `vw_customer_orders` to also include the order status column.*

```
CREATE OR REPLACE VIEW vw_customer_orders AS
SELECT c.name       AS customer_name,
       c.city,
       o.order_id,
       o.order_date,
       o.status,
       o.total
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

### Q76. [Intermediate] *CREATE INDEX*

*Create a non-clustered index on the `status` column of the `orders` table.*

```
CREATE INDEX idx_orders_status
ON orders(status);

-- Verify:
SHOW INDEX FROM orders;
```

### Q77. [Intermediate] *Composite Index*

*Create a composite index on `orders(customer_id, order_date)` to speed up queries that filter by customer and sort by date.*

```
CREATE INDEX idx_orders_cust_date
ON orders(customer_id, order_date);

-- Test with EXPLAIN:
EXPLAIN
SELECT * FROM orders
WHERE customer_id = 1
ORDER BY order_date DESC;
```

### Q78. [Intermediate] *UNIQUE INDEX*

*Create a unique index on the `email` column of the `customers` table to prevent duplicates.*

```
CREATE UNIQUE INDEX idx_customers_email
ON customers(email);

-- Verify uniqueness enforcement:
-- INSERT INTO customers VALUES(99,'Test','City','alice@example.com','2024-01-01');
-- ERROR: Duplicate entry for key 'idx_customers_email'
```

## Section 13 — DCL: GRANT & REVOKE

### Q79. [Intermediate] *CREATE USER + GRANT*

*Create a new database user `sales_rep` on localhost and grant SELECT access on all tables in `training_db`.*

```
CREATE USER 'sales_rep'@'localhost' IDENTIFIED BY 'Sales@2024';
GRANT SELECT ON training_db.* TO 'sales_rep'@'localhost';
FLUSH PRIVILEGES;

-- Verify:
SHOW GRANTS FOR 'sales_rep'@'localhost';
```

### Q80. [Intermediate] *Column-Level GRANT*

*Grant the user `intern1` SELECT access to ONLY the name and city columns of the `customers` table.*

```
GRANT SELECT (name, city)
ON training_db.customers
TO 'intern1'@'localhost';
FLUSH PRIVILEGES;
```

### Q81. [Intermediate] *REVOKE*

*Revoke the SELECT privilege on `training_db.orders` from the user `sales_rep`.*

```
REVOKE SELECT ON training_db.orders
FROM 'sales_rep'@'localhost';
FLUSH PRIVILEGES;

-- Verify:
SHOW GRANTS FOR 'sales_rep'@'localhost';
```

### Q82. [Intermediate] *GRANT via ROLE*

*Create a role `analyst_role`, grant it SELECT on all tables in training_db, then assign the role to user `alice`.*

```
CREATE ROLE 'analyst_role';
GRANT SELECT ON training_db.* TO 'analyst_role';
GRANT 'analyst_role' TO 'alice'@'localhost';
SET DEFAULT ROLE 'analyst_role' TO 'alice'@'localhost';
FLUSH PRIVILEGES;
```

## Section 14 — LIMIT & OFFSET

### Q83. [Basic] *LIMIT*

*Show the 5 most recently joined customers.*

```
SELECT customer_id, name, joined_date
FROM customers
ORDER BY joined_date DESC
LIMIT 5;
```

### Q84. [Intermediate] *LIMIT + OFFSET (Pagination)*

*Implement pagination for the products table: show page 2 with 2 products per page, ordered by price ascending.*

```
SELECT product_id, name, price
FROM products
ORDER BY price ASC
LIMIT 2 OFFSET 2;   -- Page 2: skip first 2 rows, return next 2
```

### Q85. [Intermediate] *LIMIT with Subquery*

*Find the 2nd and 3rd most expensive products using LIMIT and OFFSET.*

```
SELECT name, price
FROM products
ORDER BY price DESC
LIMIT 2 OFFSET 1;   -- skip 1 (most expensive), return next 2
```

## Section 15 — Mixed & Comprehensive Queries

### Q86. [Intermediate] *JOIN + Subquery*

*Find all customers whose latest order total is greater than ₹10,000.*

```
SELECT c.name, c.city, o.total AS latest_order_total
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.total = (
    SELECT MAX(o2.total)
    FROM orders o2
    WHERE o2.customer_id = c.customer_id
)
AND o.total > 10000
ORDER BY o.total DESC;
```

### Q87. [Intermediate] *JOIN + GROUP BY + HAVING*

*Find all customers who have spent more than ₹20,000 in total across all their orders.*

```
SELECT c.name, SUM(o.total) AS lifetime_value
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
HAVING SUM(o.total) > 20000
ORDER BY lifetime_value DESC;
```

### Q88. [Intermediate] *Correlated + EXISTS*

*List all products that have been ordered more than once (appear in multiple order_item rows).*

```
SELECT p.product_id, p.name
FROM products p
WHERE (
    SELECT COUNT(*)
    FROM order_items oi
    WHERE oi.product_id = p.product_id
) > 1;
```

### Q89. [Intermediate] *SELF JOIN + Correlated*

*For each employee, show their salary percentile within their department (what fraction of colleagues they out-earn, as a percentage).*

```
SELECT e1.name, e1.dept, e1.salary,
       ROUND(
           100.0 * (SELECT COUNT(*)
                    FROM employees e2
                    WHERE e2.dept   = e1.dept
                      AND e2.salary <= e1.salary)
           / (SELECT COUNT(*)
              FROM employees e3
              WHERE e3.dept = e1.dept),
```

```
       0) AS pct_rank
FROM employees e1
ORDER BY e1.dept, pct_rank DESC;
```

### Q90. [Intermediate] *Derived Table + JOIN*

*Find the customer who placed the highest single-order value. Show their name, city, and that order total.*

```
SELECT c.name, c.city, top_order.max_total
FROM customers c
INNER JOIN (
    SELECT customer_id, MAX(total) AS max_total
    FROM orders
    GROUP BY customer_id
) AS top_order ON c.customer_id = top_order.customer_id
ORDER BY top_order.max_total DESC
LIMIT 1;
```

### Q91. [Intermediate] *Multiple Aggregates + HAVING*

*Find products that have been ordered at least 3 times in total (across all order_items rows) AND have total quantity sold > 5.*

```
SELECT p.name,
       COUNT(oi.item_id)  AS times_ordered,
       SUM(oi.qty)        AS total_qty_sold
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name
HAVING COUNT(oi.item_id) >= 3
   AND SUM(oi.qty) > 5;
```

### Q92. [Intermediate] *UNION + Aggregate*

*Show a single result set of: (a) most expensive product name & price, and (b) least expensive product name & price.*

```
SELECT 'Most Expensive' AS label, name, price
FROM products
WHERE price = (SELECT MAX(price) FROM products)
UNION ALL
SELECT 'Least Expensive', name, price
FROM products
WHERE price = (SELECT MIN(price) FROM products);
```

### Q93. [Intermediate] *LIKE + JOIN*

*Find all orders placed by customers whose names start with 'A', along with the total amount.*

```
SELECT c.name, o.order_id, o.order_date, o.total
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE c.name LIKE 'A%'
ORDER BY o.order_date;
```

### Q94. [Intermediate] *EXISTS + JOIN*

*Find all orders that contain at least one product from the 'Electronics' category — using EXISTS.*

```
SELECT o.order_id, o.customer_id, o.total
FROM orders o
WHERE EXISTS (
    SELECT 1
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    WHERE oi.order_id = o.order_id
      AND p.category = 'Electronics'
)
ORDER BY o.total DESC;
```

### Q95. [Intermediate] *NOT EXISTS + Correlated*

*Find customers who have never ordered any product from the 'Furniture' category.*

```
SELECT c.customer_id, c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    JOIN order_items oi ON o.order_id     = oi.order_id
    JOIN products    p  ON oi.product_id  = p.product_id
    WHERE o.customer_id = c.customer_id
      AND p.category    = 'Furniture'
);
```

### Q96. [Intermediate] *View + Subquery*

*Create a view `vw_high_value_customers` that lists customers whose total spending (across all orders) is above the average spending of all customers.*

```
CREATE VIEW vw_high_value_customers AS
SELECT c.customer_id, c.name, c.city,
       SUM(o.total) AS total_spent
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
HAVING SUM(o.total) > (
    SELECT AVG(cust_total)
    FROM (
        SELECT customer_id, SUM(total) AS cust_total
        FROM orders
        GROUP BY customer_id
    ) AS t
);
```

### Q97. [Intermediate] *Correlated – Update Scenario*

*Write a SELECT that shows what each product's stock would look like after reducing it by the total quantity sold. (Read-only: no UPDATE needed.)*

```
SELECT p.product_id, p.name,
       p.stock AS current_stock,
       COALESCE((
           SELECT SUM(oi.qty)
           FROM order_items oi
           WHERE oi.product_id = p.product_id
       ), 0)   AS total_sold,
       p.stock - COALESCE((
           SELECT SUM(oi.qty)
           FROM order_items oi
           WHERE oi.product_id = p.product_id
       ), 0)   AS projected_stock
FROM products p
ORDER BY projected_stock ASC;
```

### Q98. [Intermediate] *3-Level Nested Subquery*

*Find the name of the customer who placed the order with the single highest total value, using nested subqueries only (no JOIN).*

```
SELECT name
FROM customers
WHERE customer_id = (
    SELECT customer_id
    FROM orders
    WHERE total = (
        SELECT MAX(total)
        FROM orders
    )
    LIMIT 1
);
```

### Q99. [Intermediate] *Full Scenario – Sales Report*

*Generate a complete sales report: for each customer show their name, city, number of orders, total spend, average order value, and their highest single order — for customers who have spent more than ₹10,000 in total.*

```
SELECT c.name,
       c.city,
       COUNT(o.order_id)        AS num_orders,
       SUM(o.total)             AS total_spent,
       ROUND(AVG(o.total), 2)  AS avg_order_value,
       MAX(o.total)             AS highest_order
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
HAVING SUM(o.total) > 10000
ORDER BY total_spent DESC;
```

### Q100. [Intermediate] *Comprehensive – Inventory + Sales*

*For each product, show: name, category, current stock, total units sold, projected remaining stock, and revenue generated. Sort by revenue descending. Include products with no sales (show 0 for sold/revenue).*

```
SELECT p.name,
       p.category,
       p.stock                                  AS current_stock,
       COALESCE(SUM(oi.qty), 0)                 AS total_sold,
       p.stock - COALESCE(SUM(oi.qty), 0)       AS remaining_stock,
       COALESCE(SUM(oi.qty * oi.unit_price), 0)  AS revenue
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name, p.category, p.stock
ORDER BY revenue DESC;
```